

DEVELOPING APPLICATIONS – GOOGLE APP ENGINE

Google App Engine is a cloud platform provided by Google that helps developers create, run, and deploy applications easily.

- You don't need to manage servers yourself—Google handles that.
- Google App Engine is a cloud platform for creating and deploying applications quickly and efficiently

This means:

- You can build apps faster
- You don't worry about servers, hardware, or maintenance
- Google automatically handles deployment and scaling



Building Apps with Google App Engine

• Easy Development:

- You can write simple Python code
- Make small changes to HTML
- Deploy (upload and run) your application in just a few minutes
- No complex setup is required
- This makes App Engine beginner-friendly.

• Python-Friendly:

- Knowing Python is helpful but not compulsory

- Python is: Easy to read, Similar to other scripting languages
- Even beginners can learn it quickly
- So, developers from other languages can also adapt easily.

● Java Support:

- App Engine also supports Java
- But Java: Is more complex than Python, Often costs more when hosting applications
- Hence, Python is usually preferred for small apps

Key Features of Google App Engine

1. Simple Applications:

○ Best for lightweight apps:

- Ideal for small and simple applications
- Not suitable for heavy calculations or complex processing

Examples:

Simple websites, Login systems, User profile apps.

○ Integrated with Python for basic database operations:

- Python can be used to: Store user data, Retrieve user data

Examples:

Saving usernames, Fetching email addresses. Works well for basic data handling.

2. Limitations:

These restrictions are mainly to avoid errors, especially by beginners.

○ **No direct file system access:**

- You cannot write directly to the file system
- You cannot create subthreads
- This avoids crashes and unsafe operations
- Google controls these to keep apps stable and secure.

○ **Limited database functionality:**

- ■ **Only basic operations:** You can: Search data, Store data. You cannot do complex queries.
- ■ **No support for joins:** Database joins are not allowed. That means you cannot combine multiple tables like in traditional SQL databases.
- ■ **Python syntax instead of SQL:** Database queries use Python-style syntax. Traditional SQL queries are not used. Because of this: SQL tools, Reporting tools, Graphing tools become difficult or unusable.

3. AJAX and Web Services:

- Mentioned in Google's documentation
- But in reality: Support is very minimal, Not fully powerful or flexible
- So developers should not depend heavily on these features.

Payment and Usage Limits

● **Free usage within limits:**

- Google gives free usage up to a certain level

Example:

200 million megacycles of CPU per day. Good for testing and small applications.

● **Charges after exceeding limits:**

If you go beyond free limits, you must pay. Costs may increase unexpectedly due to:

- ○ **Inter-server traffic:** Data moving between servers slows performance, Uses more resources, Increases cost
- ○ **Simultaneous users:** Many users accessing at the same time, More CPU and memory are required, This raises the bill quickly

- **Scaling:**

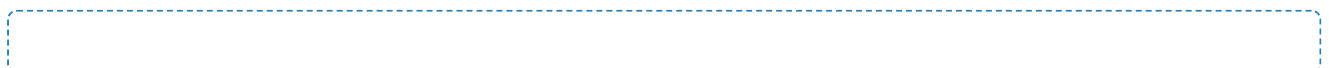
- App Engine automatically adds servers
- When user demand increases: New servers are added automatically
- Developers do not need to do anything manually

Ideal Use Cases

- • **Best for:** Small applications, Simple logic, Apps that need automatic scaling
- • **Not suitable for:** Heavy computation, Complex backend processing, Advanced database requirements

GOOGLE APP ENGINE WITH FORCE.COM

Google partnered with Salesforce.com to combine: Google App Engine + Force.com. This created a powerful cloud platform.



Force.com + Google App Engine

1. What it Offers:

- **Cloud-based development tools:** Entire application runs in the cloud. No local servers needed. Useful for: Web apps, Business apps.
- **Combined strengths:** Google handles frontend (user interface, web). Salesforce handles backend (enterprise data, CRM).

2. Benefits:

- **Scalability:** Applications can grow easily. Handles: More users, More data. No need to redesign the app.
- **Developer Support from Force.com:**
 - **Python libraries:** Developers can: Read Salesforce data, Write Salesforce data, Direct integration using Python.
 - **APIs:** APIs are provided for: Mobile applications, Analytics, User login and authentication, Security features.

3. Collaboration:

- **Shared data usage:** Apps can use: Google App Engine data, Salesforce enterprise data.

○ Example:

A consumer app on App Engine Uses enterprise customer data stored in Force.com.

4. Resources for Developers:

Developers are given: Python documentation, Sample Python code, Testing tools, FAQs and best practices. This makes learning and development easier.

GOOGLE GEARS

Google Gears was an open-source browser extension. Its main purpose: Allow web apps to work offline.

Key Features of Google Gears

1. Offline Functionality:

- Applications can work: Without internet, With poor or unstable connections

○ Example:

Google Reader used Gears. Users could read content even without internet.

2. APIs:

Google Gears introduced JavaScript APIs for:

- ■ **Data storage:** Store data locally on the user's device
- ■ **Application caching:** Save web pages locally, Load them faster
- ■ **Multithreading:** Run background tasks, Improve performance

3. Cross-Platform Support:

- Works on: Chrome, Firefox, Opera
- Works on: Windows, Mac, Linux
- This made it widely usable.

4. Goal:

Push the web industry toward: A single offline web standard. Inspired future offline technologies.

DEVELOPING APPLICATIONS – MICROSOFT AZURE

Microsoft Azure is a complete cloud platform for developers.

What is Azure?

- **Cloud platform in Microsoft data centers:** Applications run on Microsoft's servers. No need for local infrastructure.
- **Build cloud applications:** Apps can be: Fully cloud-based, Partially cloud-based.
- **On-premises integration:** Azure can connect with: Local company servers. Useful for hybrid environments.

Key Features of Azure

1. **Infrastructure:** High availability (always running), Dynamic scaling based on usage, Pay only for what you use.
2. **Developer Tools:** Integrated with: Visual Studio, .NET Framework. Supports many standards: HTTP, REST, SOAP, XML.
3. **Applications:** You can build apps for: Web, Mobile, PCs, Servers, Hybrid (cloud + on-premises).
4. **Microsoft Cloud Applications:** Azure hosts ready-made apps like: Windows Live, Microsoft Dynamics, SharePoint Online, Exchange Online.

Azure Services

- **1. Live Services:** Manage user data, Build social applications, Work across multiple devices.
- **2. Microsoft SQL Services:** Cloud version of SQL Server. Supports: Relational queries, Search, Data synchronization.
- **3. Microsoft .NET Services:** Allows loosely coupled applications. Includes:
 - **Access Control:** Secure user access
 - **Service Bus:** Enables communication between apps
 - **Workflow Execution:** Manages cloud workflows

- **4. SharePoint and Dynamics CRM Services:** Collaboration tools, CRM tools, Integrated using Visual Studio.

Azure Architecture

Azure follows a layered architecture.

- **Layer Zero – Global Foundational Services (GFS):** Works like a hardware abstraction layer. Communicates directly with servers.
- **Layer One – RedDog:** This is the Azure Operating System. It manages: Storage, Fabric Controller (resource allocation), Virtual machines, Local development emulator.
- **Layer Two – Building Blocks:** Provides services like LiveMesh. Developers build apps using these services.
- **Layer Three – Applications:** Microsoft apps, Third-party apps. Examples: SharePoint Online, Dynamics CRM, Exchange Online.

Scaling with Azure

Azure supports two types of scaling.

1. Vertical Scaling (Scale-Up):

- Add more CPU or RAM to one system
- Best for: Memory-intensive apps, CPU-heavy apps
- Limited by hardware capacity

2. Horizontal Scaling (Scale-Out):

- Add more servers
- Best for: Web applications, Distributed systems
- Reduces I/O bottlenecks

Trade-Offs: Vertical scaling: Simple, Hardware-limited. Horizontal scaling: More complex to manage, More cost-effective at large scale.

INTUIT QUICKBASE

Intuit QuickBase is a low-code / no-code cloud platform. Its main purpose is to help people create business applications easily, even if they do not have strong technical or programming skills. This platform is mainly designed for business users, not hardcore programmers.

Key Features of Intuit QuickBase

1. Ease of Use

- ○ **Build or customize apps without coding expertise:** Business consultants and companies can create or modify applications. No deep programming knowledge is required. This reduces dependency on IT teams.
- ○ **Over 200 templates available:** Ready-made templates are provided. Users can quickly customize them instead of starting from scratch. Saves time and effort in application development.

2. Efficiency

- ○ **Faster development with better returns:** Applications can be built: In one-fourth the time, At half the cost, With double the profit margins. Compared to traditional software development, this is much more efficient.
- ○ **SaaS implementation advantages:** Since QuickBase is delivered as Software as a Service (SaaS): Deployment is faster, Maintenance cost is lower, No infrastructure setup is needed.

3. Business Consultant Program

- ○ **Support for consultants:** Provides: Training programs, Lead-generation tools, Partner relationship management. Helps consultants grow their business using QuickBase.

- ○ **Free QuickBooks Online:** Consultants get a free version of QuickBooks Online. Helps them manage: Accounting, Billing, Their own business operations.

4. Target Audience

- ○ **Designed for:** Entrepreneurs, Industry experts, Business process consultants.
- ○ **Shift in software development:** Software customization moves: From IT professionals, To domain specialists (people who understand the business). This improves accuracy and relevance of applications.

CAST IRON CLOUD

Cast Iron Cloud is an integration platform. It connects SaaS applications with enterprise and legacy systems. This model is called Integration as a Service (IaaS).



Key Features of Cast Iron Cloud

1. Integration Flexibility

- ○ **Connects different system types:** Cloud-based systems, On-premise systems, Legacy systems (old systems). All can be connected efficiently.
- ○ **Deployment options:** Cloud-based integration, On-premise appliances. Organizations can choose based on their needs.

2. Prebuilt Templates

- ○ **Template Integration Processes (TIPs):** Ready-made integration templates. Designed for common SaaS integration use cases.
- ○ **Quick deployment:** TIPs can be deployed in minutes. No custom coding required. Ideal for fast integration projects.

3. Benefits

- ○ **No infrastructure investment:** No need to buy servers. No need for middleware expertise.
- ○ **Simplified operations:** Easier data integration, Simplified workflows, Real-time monitoring of processes.

4. Applications

○ **Common integrations:** Often used with: Salesforce CRM, NetSuite ERP. Helps connect modern cloud systems with older enterprise systems.

5. Self-Guided Wizard

- ○ **Easy customization:** Non-technical users can modify TIPs. Uses a wizard-based approach.
- ○ **Example comparison:** Similar to how TurboTax guides users step-by-step. Makes complex integration simple.

BUNGEE CONNECT

Bungee Connect is a cloud platform used to: Develop, Deploy, Host web applications. It runs on a multitenant grid infrastructure, meaning many users share the same system safely.

Key Features of Bungee Connect

1. All-in-One Development

- ○ **Single platform for everything:** Development, Testing, Deployment, Hosting. All are handled in one environment.
- ○ **Faster time-to-market:** Reduces development time by up to 80%. Applications reach users much faster.

2. Cloud-Based Functionality

- ○ **Browser-only access:** Works entirely through a web browser. No downloads or plug-ins needed.
- ○ **Strong performance and security:** Supports highly interactive applications. Provides robust security features.

3. Automated Integration

- ○ **Web services integration:** Supports: SOAP services, REST services.
- ○ **Database support:** Integrates with databases like: MySQL, PostgreSQL.

4. Utility Pricing Model

- ○ **Pay for usage:** Cost depends on how much the app is used. No fixed heavy investment.
- ○ **Pricing range:** \$2 to \$5 per user per month. Suitable for business applications.

5. Collaboration Tools

- ○ **Team collaboration:** Built-in tools for teamwork. Helps multiple developers work together.
- ○ **Deep instrumentation:** Tracks: App performance, Usage patterns. Useful for optimization and analytics.

DEVELOPING APPLICATIONS ON GOOGLE APP ENGINE

Google App Engine is a Platform as a Service (PaaS). It allows developers to build and deploy applications easily, mainly using Python.

Getting Started with Google App Engine

1. Prerequisites

- ○ **Required installations:** Python 2.5, Google App Engine SDK.
- ○ **SDK features:** Local development server, Tools for uploading apps to the cloud.

2. Developing an App

- ○ **CGI standards:** Apps follow CGI (Common Gateway Interface). Used to handle: Web requests, Web responses.
- ○ **Example process:**
 - Create a directory (example: heresjohnny)
 - Write a Python file (heresjohnny.py)
 - The script: Sends an HTTP header, Displays a message to the user

3. Configuration File (app.yaml)

- **Purpose of app.yaml:** Configures how the app runs. Defines routing and settings.

○ **Example fields explained:**

- **Application ID:** Unique name of the app
- **Version:** Application version number
- **Runtime:** Programming language used
- **Handlers:** Maps URLs to scripts

4. Testing and Deployment

- ○ **Local testing:** Use SDK's development server. Test before going live.
- ○ **Deployment:** Upload using appcfg.py. Google hosts the app automatically.
- ○ **Accessing the app:** Apps are available at: <http://application-id.appspot.com>

SALESFORCE.COM DEVELOPMENT (USING SALESFORCE.COM)

Salesforce provides a cloud-based platform where you can build applications using point-and-click tools, without writing code.

This section explains how to build a simple Lunch Expense Tracking application.

1. Create an Account

- **Go to developer.force.com:** This is Salesforce's official website for developers. You sign up for a Developer Edition account, which is: Free, Fully functional, Used for learning and testing.
- **Complete the setup process:** You must enter: Your name, Username, Password, Email and other basic details. Salesforce creates a cloud environment for you.
- **Log in at login.salesforce.com:** After registration, you access your account through this login page. This is where all development work is done.

2. Create a Custom Object

A custom object is like a table in a database. It stores data that is specific to your application.

Steps to create a custom object:

1. **Go to Setup:** Click the Setup link at the top-right corner. Setup is where all configuration happens.
2. **Navigate to Create → Objects:** This section allows you to create new data structures. Click New Custom Object.
3. **Fill in the details:**
 - **Label:** Lunch → Name shown to users
 - **Plural Label:** Lunch → Used when multiple records exist

- **Object Name:** Lunch → Internal system name
 - **Description:** → “An object that holds lunch expense information.” This clearly defines the purpose of the object.
4. **Check options:** Allow Activities (Enables tasks and calendar events), Allow Reports (Enables reporting on this object).
 5. **Save the object:** Salesforce now creates the Lunch object. This object will store lunch expense data.

3. Add Fields

Fields define what information is stored inside the Lunch object. Examples: date, cost, and contact.

Step 1: Add a Date Field

- **Go to Custom Fields & Relationships:** This section manages object attributes.
- **Click New:** Starts the field creation wizard.
- **Choose Data Type: Date**
- **Fill in the following:**
 - Field Label: Date
 - Field Name: Date
 - Description: Date of lunch
 - Default Value: Today() → Automatically sets today's date
- **Mark as Required:** User must enter this value. Prevents incomplete records.
- **Save the field**

Step 2: Add a Cost Field

- **Choose Data Type: Number**
- **Fill in:**
 - Field Label: Cost
 - Length: 4

- Decimal Places: 2
- Field Name: Cost
- Description: Cost of lunch
- • **Mark as Required:** Ensures every lunch has a cost.
- • **Save the field**

Step 3: Add a Contact Field

- • **Choose Data Type: Lookup Relationship:** This creates a link to another object.
- • **Related To: Contact:** Links lunch expense to a person.
- • **Fill in details:**
 - Field Label: Contact
 - Field Name: Contact
 - Description: Person I had lunch with
- • **Save the field**

4. Create a Tab

Tabs make objects visible and accessible to users.

Steps:

1. Go to Setup → Create → Tabs
2. Click New under Custom Object Tabs
3. Select the Lunch object
4. Choose a tab style (example: Apple icon)
5. Accept default settings
6. Save

Now users can click the Lunch tab.

5. Build the App

An app groups related objects, tabs, and pages together.

Steps:

1. Go to Setup → Create → Apps → New
2. Enter app details: App Label: Lunch Tracker, App Name: Lunch_Tracker, Description: Tracks lunch expenses.
3. Add tabs: Add the Lunch tab, Set Home tab as default page.
4. Make the app visible: Mark as Visible to all users.
5. Save the app.

6. Test the App

- **Select Lunch Tracker from App Menu:** Found at the top-right corner.
- **Click the Lunch tab:** Create a new lunch record.
- **Enter details:** Date: Lunch date, Cost: Expense amount, Contact: Lookup or create a contact (Example: Bruce Dickinson).
- **Save the record:** Confirms the app works correctly.

Summary (Salesforce Section)

Using Salesforce's point-and-click tools, you:

- Created a custom object (Lunch)
- Added date, cost, and contact fields
- Created a custom tab
- Built a Lunch Tracker app
- Tested everything without writing any code

WHAT IS MICROSOFT AZURE?

Microsoft Azure is a cloud computing platform by Microsoft.

- • **What Azure allows:** Create applications, Deploy applications, Manage applications in the cloud.
- • **Core capabilities:** On-demand computing power, Cloud storage, Tools for scaling web apps automatically.

Getting Started with Azure Development

1. Download the Azure SDK

- • **Azure SDK:** A collection of tools needed to develop Azure apps. Downloaded from Microsoft's Azure SDK website.
- • **System requirements:**
 - Supported operating systems: Windows Server 2008, Windows Vista (Business / Ultimate).
 - Required software: .NET Framework 3.5 SP1, IIS 7.0 with ASP.NET and WCF HTTP Activation, Microsoft SQL Server Express 2005 or 2008, Windows PowerShell (optional).

2. Install the SDK

- • **Remove old SDK versions:** Prevents conflicts.
- • **SDK components:**
 - ○ **Development Fabric:** Simulates Azure runtime locally. Allows testing before deployment.
 - ○ **Development Storage:** Emulates cloud storage: Blob storage, Queue storage, Table storage.

Enabling IIS 7.0

Azure apps require IIS (Internet Information Services).

For Windows Server 2008: Open Server Manager, Add .NET Framework 3.0, Enable WCF HTTP Activation. Install Web Server (IIS) with: Static Content, ASP.NET.

For Windows Vista: Control Panel → Programs → Programs and Features. Turn Windows Features On or Off. Enable: WCF HTTP Activation, ASP.NET, Static Content under IIS.

Testing the SDK

- Open Azure SDK Command Prompt
- Navigate to sample directory
- Run: RunDevStore.cmd → sets up storage, runme.cmd → launches sample app
- **Result:** Sample app (Hello World) runs locally. Development Fabric icon appears in system tray.

Creating an Azure Application

Using Visual Studio 2008:

Steps:

1. Run Visual Studio as Administrator
2. File → New → Project
3. Choose Cloud Services
4. Select Web Cloud Service
5. Create a Web Role
6. Add custom code: Modify Default.aspx, Display custom text

Testing Locally

Press F5 → run app. Use Development Fabric to: View logs, Monitor app. Press SHIFT + F5 → stop debugging.

Publishing to the Azure Cloud

Open Azure Management Portal. Upload: Application files, Configuration settings. Azure deploys the app to the cloud.

Application Management

After deployment, tools like Kaavo IMOD are used.

Management capabilities:

- ○ **Backup and security:** Schedule automatic backups, Encrypt data.
- ○ **Performance monitoring:** Set alerts for CPU, memory, usage.
- ○ **Scaling:** Increase capacity during peak demand.

Troubleshooting Cloud Apps

1. Challenges:

- Less visibility than on-premise systems
- Hard to distinguish: SaaS issues, Network issues

2. Solutions:

- Use monitoring tools
- Analyze traffic patterns
- Detect: Bandwidth bottlenecks, Connectivity problems

Key Features of Azure Development

- Development Fabric: Local testing
- Visual Studio Integration: Seamless development
- Scalability: Horizontal (add nodes), Vertical (add resources)
- Management Tools: Monitoring, troubleshooting, scaling